# darkmpc, Milestone 3 walkthrough, part 1

The Stage 1 auction in MPC

Tave La Rocque

4 November 2026

## Milestone 3, part 1: the Stage 1 auction in MPC

### What was built

The crate `darkmpc-auction` runs the Stage 1 mechanism on secret-shared orders. No server ever learns any order's symbol, side, size or minimum fill. It is built from two additions to the core. The first is oblivious permutation and stable sorting: a shuffle, applying a secret permutation, and radix sort, following Asharov et al. (CCS 2022). The second is segmented prefix sums. Across 3,000 random batches for each of two generators, plus the hand-worked cases, the MPC auction returns exactly the fills of the reference matcher run with the same fixed pass count. The batches range from 1 to 48 orders, use 1 to 5 elimination passes, and cover both a spread of symbols and deep books in only two. Ten planted bugs across the three crates are all caught. The workspace has 47 passing tests.

The security model is still semi-honest with an honest majority. The servers still run as threads in one process. Separate processes, real network latency and the cloud benchmarks are part 2.

### Moving data without revealing where it goes

The radix sort and the final routing of fills both need to move secret values to secret positions. The building block is a shuffle by a random permutation that no single server knows. Write it as $\pi = \pi_2 \circ \pi_1 \circ \pi_0$, where piece $\pi_c$ is known to every server except server $c$. Each piece is generated from the pseudorandom stream the two servers in that pair already share, so drawing a random permutation needs no communication. Every server misses one piece, so no server knows $\pi$.

Applying piece $\pi_c$ is one reshare step. The two servers that know it, call them A and B, between them hold the secret as a two-party sum. A's part is its two summands added together, and B's part is the summand A lacks. Each permutes its own part locally. They must then rebuild a three-party sharing that includes the excluded server C. With $r$ and $m$ drawn from their shared stream, the new summands are

$$y_{c+2} = r, \qquad y_{c+1} = a' - r + m, \qquad y_c = b' - m.$$

Their sum is $a' + b'$, the permuted secret. A keeps $(y_{c+1}, r)$ and sends $y_{c+1}$ to C. B keeps $(r, y_c)$ and sends $y_c$ to C.

The mask $m$ deserves a sentence, because an earlier draft of my design left it out. B's part $b$ is the summand that A lacks, and C holds that summand too. Without $m$, C would receive $b' = \pi_c(b)$, a permutation of a vector it already knows, and could read off $\pi_c$ by matching values. With $m$, each value C receives is uniformly random, and their sum is masked by $r$, which C never sees.

A shuffle is three such steps, one per piece. It costs three rounds, and each server sends one word per element in the two steps where it is a pair member. A test pins that cost. Undoing a shuffle runs the same steps in reverse order with the inverse pieces.

To apply a secret permutation $\rho$, the servers shuffle both $\rho$ and the data by the same fresh random $\pi$, then open the shuffled $\rho$. What they see is $\rho \circ \pi^{-1}$. Because no server knows $\pi$, that is a uniformly random permutation from each server's point of view, so it says nothing about $\rho$. They then apply it to the shuffled data in the clear, and the shuffle and the reordering cancel into $\rho$. Applying the inverse works the same way with an unshuffle at the end.

## Sorting without comparisons

To sort by one key bit $k$, stably, put the zeros first and the ones after them, each group in its original order. With $s_i^{(0)}$ the number of zeros among the first $i$ elements, $Z$ the total number of zeros, and $s^{(1)}_i = Z + $ the number of ones among the first $i$, element $i$ goes to position

$$s_i^{(0)} + k_i \left(s_i^{(1)} - s_i^{(0)}\right) - 1 \quad \text{(counting from 0).}$$

If $k_i = 0$, this is the element's rank among the zeros. If $k_i = 1$, it is $Z$ plus its rank among the ones. Every prefix count is a local sum of shares, so the whole step costs one multiplication per element. The paper's worked example, bits $(1, 1, 0, 0)$ giving positions $(2, 3, 0, 1)$, is a test.

Radix sort repeats this from the least significant key bit to the most significant, applying each bit's permutation before computing the next. Because each step is stable, the result is sorted by the whole key, and equal keys keep their arrival order. darkmpc's key puts the side bit lowest and the 13 symbol bits above it, so the sorted batch is grouped by symbol, with buys before sells inside each symbol and each side in arrival order. That is exactly the layout the time-priority rule needs. The original index travels through the sort as one more column. Its final value at each position, the *origin*, is what routes each fill back to its order at the end.

One simplification differs from the paper. Instead of composing the per-bit permutations, darkmpc carries the remaining key bits and the payload through every bit's permutation. That needs only shuffle, open and local reordering, so it is easier to get right. It costs more communication, and the measurements below show that cost.

## Sums inside a symbol

With the batch sorted, position $t$ starts a new symbol exactly when its symbol is larger than its predecessor's. That takes one comparison per position, and the result is a shared flag. A segmented prefix sum adds values up but restarts at every flag. In the Hillis-Steele pattern, at distances $d = 1, 2, 4, \ldots$, every position does

$$v_t \leftarrow v_t + (1 - F_t)\, v_{t-d}, \qquad F_t \leftarrow F_t + F_{t-d} - F_t F_{t-d},$$

where $F_t$ records whether a segment start has been seen in the window so far. After $\lceil \log_2 n \rceil$ rounds each position holds the sum of its segment up to itself. Running the same scan on the reversed sequence gives the sum from each position to the end of its segment. Adding the two and subtracting the element itself gives the segment's total at every position.

That is all the matcher needs. The symbol's eligible buy total $B$ and sell total $S$ are segment totals. Because buys precede sells within a symbol, the eligible volume ahead of an order on its own side is the forward sum on that side minus the order itself. Each elimination pass then evaluates the reference's formulas with the core's comparisons:

$$T = \min(B, S), \qquad \mathit{fill} = \min\left(\mathit{size}, \max(0, T - \mathit{ahead})\right), \qquad \mathit{viol} = [0 < \mathit{fill}] \cdot [\mathit{fill} < \mathit{min}].$$

A violator's eligibility drops to zero for the next pass. After the last pass, a segmented total of the violation flags tells each position whether its symbol is still unstable, and the fail-safe zeroes those fills. Finally, applying the origin permutation sends each fill back to its order's slot.

## Measured cost (in-process, no network)

From `cargo run --release -p darkmpc-auction --example cost`. The batch is random, with 13-bit symbols spread over 8,192 symbols, and the servers run as threads on one machine:

| Orders | Passes | Rounds | Data sent per server | Wall time |
|---|---|---|---|---|
| 1,000 | 1 | 182 | 4.7 MB (4.6 KiB/order) | 0.017 s |
| 1,000 | 3 | 316 | 6.9 MB (6.7 KiB/order) | 0.026 s |
| 8,192 | 3 | 340 | 60.6 MB (7.2 KiB/order) | 0.216 s |
| 32,768 | 3 | 356 | 253.8 MB (7.6 KiB/order) | 1.014 s |

Wall time here is compute only. With no network latency, it says nothing about real deployment; part 2 measures that. Two numbers matter now. Rounds grow only with $\log n$: from 1,000 to 32,768 orders they go from 316 to 356. Communication is about 7 KiB per order per server, and most of it is the sort carrying every column through 14 bit-steps.

For context only, not as a comparison: the 2018 paper's volume match takes 4,509 rounds for 1,000 orders. The settings differ in every direction. Theirs is actively

secure, single-instrument, has no minimum fills, and reveals nothing about symbols because there is one. Ours is semi-honest, hides symbols and sides, and handles minimum fills. The fair comparison needs the malicious version of ours, Milestone 4, run in their setting.

## Obvious optimisations, deliberately not yet made

Four optimisations are deliberately left for later, in order of expected payoff. The paper's permutation composition would replace carrying every column through the sort, which is most of the bytes. The forward and backward scans are independent, so running them in the same rounds would save $\log_2 n$ rounds per pass. Independent comparisons can be batched into one call. And per-bit comparison costs could be cut by packing. None changes what is computed, and the differential tests stay the safety net when they are made.

## Planted bugs added in this milestone

**An unstable sort** places the ones of a key bit in reverse arrival order. It is still a valid permutation, so every value still arrives somewhere sensible, but time priority inside a group breaks. The sort test catches it, and so do the differential tests.

**A segmented scan that ignores segment boundaries** lets one symbol's volume leak into the next symbol's totals.

**A missing unsort** returns fills in sorted order rather than routing them back to their orders.

Each is caught. `check_planted.sh` now builds every planted bug against the whole workspace, so a test in any crate can catch a bug in any other.